Interactive Texture Segmentation of 3-D Scanned Models Leveraging Multiview Automatic Segmentation

Koki Madono, *Waseda University/Preferred Networks, Tokyo, 169-8555, Japan*

Takeo Igarashi, Hiroharu Kato, Taisuke Hashimoto, Fabrice Matulic, Tsukasa Takagi, and Keita Higuchi, *Preferred Networks, Tokyo, 100-0004, Japan*

In 3-D model scanning, the raw texture of a 3-D model often requires segmentation into distinct regions to apply different material properties to each region. Current methods, such as manual segmentation, are labor-intensive, while automatic segmentation techniques lack user control. We propose an interactive tool that combines automatic segmentation with minimal manual intervention, striking an optimal balance between efficiency and control. Following a multiview automatic segmentation process that divides the texture into small subsegments, users cluster the subsegments into segments by drawing simple scribbles in the 3-D model view. We show that our approach results in more detailed subsegments compared to automatic segmentation approaches. Furthermore, a user study confirms that our approach improves segmentation accuracy and quality compared to manual segmentation with standard 3-D computer graphics software. This research paves the way for more efficient texture segmentation in 3-D model scanning.

Advancements in computer technology have made high-quality 3-D scanning accessible to a wide range of users. Modern reconstruction techniques based on photogrammetry using RGB cameras produce polygonal meshes together with UV-mapped texture atlases. As these "3-D-scanned models" become ubiquitous, postprocessing is essential: unlike artist-created 3-D meshes (e.g., Blender[a] and ZBrush[b]), users have little control over the raw mesh and its textures, which often exhibit disconnected UV islands, irregular parameterization (nonuniform texel density), and small misalignments or noise. To edit 3-D-scanned models, we often require user-desired segmentation strategies in postprocessing to enable reliable subcomponent editing—altering colors, materials, or shape.

A straightforward method for segmenting 3-D models involves users manually partitioning the texture using 3-D software. For example, Blender offers basic 3-D painting functionality, such as paint brushes and fill tools, which can facilitate 3-D model segmentation. However, this manual segmentation process is labor-intensive and requires using these 3-D painting tools with meticulous precision.

On the other hand, automatic segmentation solutions aim to eliminate the need for user involvement. However, heuristic techniques, such as k-means clustering[1] and superpixel segmentation,[2] struggle to accurately identify useful segments based on both 3-D shape and texture information. More advanced techniques using deep learning demonstrate superior capabilities in automatically extracting meaningful segments,[3,4,5] but these methods do not guarantee results that align with users' needs or application-specific requirements. Despite these technological advancements, the wide range of user preferences and the subjective nature of

[a][Online]. Available: https://www.blender.org
[b][Online]. Available: https://www.maxon.net/ja/zbrush

0272-1716 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.
This article has supplementary downloadable material available at https://doi.org/10.1109/MCG.2025.3595378, provided by the authors.
Digital Object Identifier 10.1109/MCG.2025.3595378
Date of publication 4 August 2025; date of current version 26 May 2026.

Authorized licensed use limited to: University of Waterloo. Downloaded on May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.

ideal segmentation present challenges for solutions solely relying on automated processes. This highlights the need for adaptable, user-driven techniques to adequately address the requirements of applications involving 3-D scanned models.

We propose a semiautomatic interactive approach to texture segmentation, addressing the limitations of both fully manual and fully automatic methods. Our technique aims not only to reduce the labor-intensive efforts associated with manual segmentation but also to give users greater control over automatic processes. In addition, efficient manual input is crucial for the creation of annotated datasets to improve automatic segmentation. Given the diverse needs and preferences of users, even for the same 3-D model, supporting flexible, tailored, and effective segmentation is essential to produce high-quality 3-D models.

Our technique first involves precomputing texture image segmentations and multiview-rendered image segmentations of the 3-D scanned model using geometric features that are not contained in the texture image. From these results, boundaries are extracted and textures are split into small subsegments, which serve as segment candidates. In a subsequent interaction phase, users add color scribbles on the 3-D view of the scanned model to guide the subsegment-selection process for segmentation. Specifically, when a color scribble is added, a segmentation ID is assigned to subsegments intersected by the scribble, and the ID is propagated to nearby subsegments to generate a segmentation. This process is repeated with the user adding or removing scribbles to iteratively refine the segmentation according to their needs.

The development presented in the work of Madono et al.[6] forms the foundation of our study, demonstrating that our method achieves superior segmentation accuracy, task completion speed, and overall user experience compared to conventional manual-painting interfaces. In this article, we extend our prior work[6] by performing a comprehensive quantitative evaluation of the proposed segmentation approach on several publicly available 3-D-scanned models. Our primary contributions are as follows.

- We conduct a thorough comparison of the proposed segmentation algorithm against recent state-of-the-art methods with publicly available 3-D models, confirming significant improvements in accuracy.
- We evaluate the importance of using both 3-D views of segmentation and visibility-aware camera sampling.

RELATED WORK

Automatic 3-D Segmentation

Early automatic 3-D segmentation techniques involved heuristic approaches, such as k-means[1] and graph cuts.[7] More recently, the segment anything model (HQ-SAM)[3] has achieved significant success in producing high-quality segmentations of 2-D images. To adapt SAM to 3-D content, there are mainly three approaches: matching-based,[4] neural radiance field-based,[8] and Gaussian-splatting-based[9] methods.

Compared with the approaches taken in those previous works, which typically involve generating meshes from multiple images, we operate under different conditions, as we work with 3-D scanned models and their corresponding UV textures. SAM3D[4] has a similar problem setting with segments using a 3-D scanned model. However, unifying the multiple 2-D segmentation results into one consistent 3-D segmentation is not straightforward. Our approach applies 2-D segmentation to texture images *as well as* multiview 3-D rendered images in order to create subsegments for flexible, user-driven texture segmentation of 3-D scanned models.

Interactive Segmentation

Technical Approaches to Support Interactivity

Various interactive segmentation techniques have been proposed. For example, Igarashi et al.[10] proposed an interactive 3-D volume segmentation approach using value thresholding, and Kawabe et al.[11] presented a user interface that uses the k-means algorithm to segment material surfaces with sliders to allow the user to adjust hyperparameters. Heuristic methods can be easily adapted to several types of input data; however, finding good hyperparameters is difficult, and in some cases, accuracy is quite low.

To address the limitations of heuristic approaches for interactive segmentation, machine learning techniques have been considered with different strategies for user interaction. Click-based interactions, such as segment anything in high quality,[3] achieve high-quality segmentation; however, studies report that they often require a large number of clicks,[12] resulting in significant user effort and making it difficult to produce robust segmentations, particularly when tracing boundaries or working with regions with low color contrast. Color adjustment[13] is another promising approach for image segmentation. However, it is often challenging to fine-tune contrast effectively, leading to noisy outputs in many cases.

While these works use scribble-based segmentation, it has been widely used in interactive segmentation

May 2026 Authorized licensed use limited to: University of Waterloo. Downloaded on May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.

tasks, e.g., GraphCut[14] and LazyBrush.[15] These works highlight the intuitiveness and efficiency of simple strokes to guide semiautomatic editing processes. Furthermore, scribble-based techniques enable users to easily distinguish both boundaries and regions, even in low-contrast conditions. Our technique improves on these methods by generating subsegments and calculating their relationships beforehand in order to reduce the number of scribbles input by the user to produce high-quality segmentations.

Manual Segmentation With 3-D Software

Interactivity plays a crucial role in user-driven 3-D surface segmentation. Recent techniques attempt to alleviate users' burden through various approaches.

Popular 3-D paint software, such as Blender[a] and ZBrush,[b] enhances interactivity by integrating numerous convenient tools within their interfaces (e.g., bucket and brush tools), to directly paint on the UV map. These interfaces further provide a dual view of both the 3-D model and the UV map during segmentation to facilitate painting operations. However, these solutions require a certain level of practice and proficiency, potentially making them less accessible to novice users.

To achieve easy-to-use, efficient interactive segmentation, we propose an approach that first segments the texture into a multitude of small subsegments and then gradually merges these regions based on user-provided scribbles. This approach strikes a balance between automation and user control, allowing for precise and intuitive segmentation.

USER INTERFACE

Target Problem Setting

Our research question is how to leverage the efficiency of modern automatic segmentation algorithms while retaining some of the flexibility and control afforded by manual techniques to segment 3-D scanned models for which 3-D mesh and UV texture data are available. The resulting segmentation data can then be used to edit material and texture properties of these 3-D models, as illustrated in Figure 1.

While manual segmentation affords full control to precisely assign class labels to each UV pixel, this process is tedious and time-consuming. In practice, certain areas lend themselves to straightforward classification into the same category due to uniform colors or shapes. Conversely, other regions with nonuniform textures present challenges.

To mitigate the need for manual labor, we propose a hybrid segmentation approach: automating the segmentation of easily identifiable regions and enabling user-guided segmentation for more complex areas. To achieve this, we decompose the UV texture into subsegments of varying sizes and merge these subsegments into segments based on scribbles (strokes) subsequently input by the user. This approach aims to streamline the texture segmentation process, enhancing flexibility and efficiency when handling 3-D scanned models.

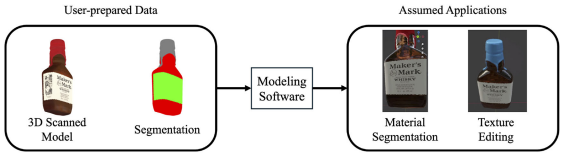


FIGURE 1. Applications for editing 3-D scanned models using segmentation information.

Design Rationale and Tools

Figure 2 shows a screenshot of our interactive texture segmentation interface. Our system aims to be straightforward and intuitive for novice users, and therefore, we create simple and intuitive tools for our interface.

The UI consists mainly of two panels showing the 3-D model with its original texture (textured view) and the model with the current color-coded segmentation results (segmentation view) (see Figure 2). The two

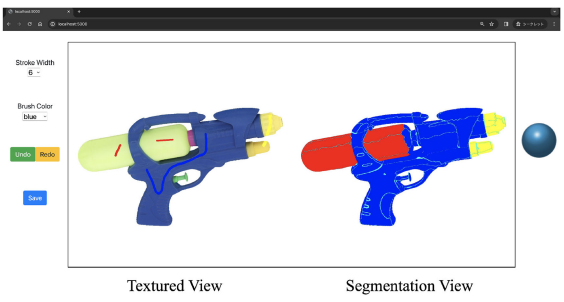


FIGURE 2. Screenshot of our system. It consists of brush adjustment (brush size and color), undo/redo operation, save operation of UV map, and camera control. On the main screen, the user can simultaneously check both textured mesh and segmentation mesh, which always visualize the boundaries by light blue color. Once the scribble is added to a textured view, the segmentation results are updated. In the figure's example, the user added three scribbles (red, blue, and yellow), from which the segmentation is automatically estimated.

Authorized licensed use limited to: University of Waterloo. Downloaded on May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.

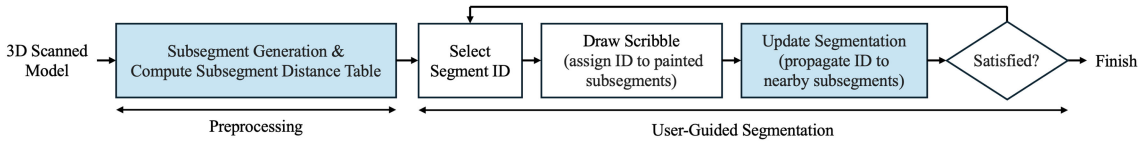


FIGURE 3. Overview of segmentation workflow on our interface. The user iteratively provides scribbles to the 3-D model and evaluates the automatic segmentation results until achieving satisfaction. The user workflow part is indicated by a white box and machine workflow is indicated by a blue box.

views can be simultaneously panned, rotated, and scaled using the shift button, the camera control button on the right, and mouse, respectively.

After choosing a color representing the ID of a particular region to segment, the user draws short strokes (scribbles) with the mouse on the corresponding parts of either view (textured or segmentation view). In the background, the system assigns the selected segment ID to the subsegments intersected by the scribble and then propagates this segment ID to neighboring subsegments to generate a segmentation. The segmentation view is subsequently updated to reflect the new segmentation. This procedure is repeated, where the user chooses a different color to mark other regions of the model.

At each stage, the user can change the size of the brush as well as undo/redo operations to facilitate the scribbling process.

When the user is satisfied with the results, they can save the UV map (to be used in a 3-D application to further edit material and texture properties).

User-Guided Segmentation Workflow

Our segmentation technique, as illustrated in Figure 3, consists mainly of the following two steps.

1) *Preprocessing:* Automatic subsegment generation and computation of subsegment distance table.
2) *User-Guided Segmentation:* The user iteratively adds scribbles on the UI, upon which segmented regions are updated, until the results are satisfactory.

We assume that subsegments and a subsegment distance table have been computed in the preprocessing stage. In the computation, hyperparameters were adjusted to generate a reasonable number of subsegments. In practice, we find that 100–1000 subsegments strike a good compromise between preprocessing computational costs and the level of subsegment granularity required to support detailed segmentations.

Segmentation Example

Figure 4 shows an example of the interactive segmentation process with a figurine.

The process begins with a model obtained from scanning the physical figurine and computing subsegments using the algorithm detailed in the "Preprocessing" section. After subsegmentation is complete, the user adds scribbles with the interface to perform the actual segmentation. In our example, the user starts by adding red scribbles to cluster the first set of features of the figurine: its mouth and ear. Since these are the initial scribbles, all subsegments are colored in red. Next, the user draws blue scribbles for the black elements of the figurine, which triggers a recalculation of segment boundaries using the algorithm described in the "User-Guided Subsegment Clustering" section. After this second iteration, the mouth and ear remain grouped under the same category (and therefore in red color), while the face surface becomes a distinct blue segment. The user then proceeds to add further scribbles in different colors for the other regions of the object until they are satisfied with the results.

With this iterative process, users can progressively refine the segmentation, while leveraging the system's automatic segment clustering. At any time, the user can rotate the camera view of the 3-D model to confirm and adjust the current segmentation for all parts of the model.

IMPLEMENTATION

Overview

During the preprocessing phase, we calculate subsegments and a subsegment distance table that records the distances between subsegments. These subsegments are utilized for interactive segmentation, enabling users to adaptively adjust the segmentation results, a flexibility that is not provided by fully automatic segmentation approaches. The precomputation of the distance table aids in accelerating the interactive segmentation process.

Figure 5 shows an overview of the preprocessing step. The details of each algorithm are described in

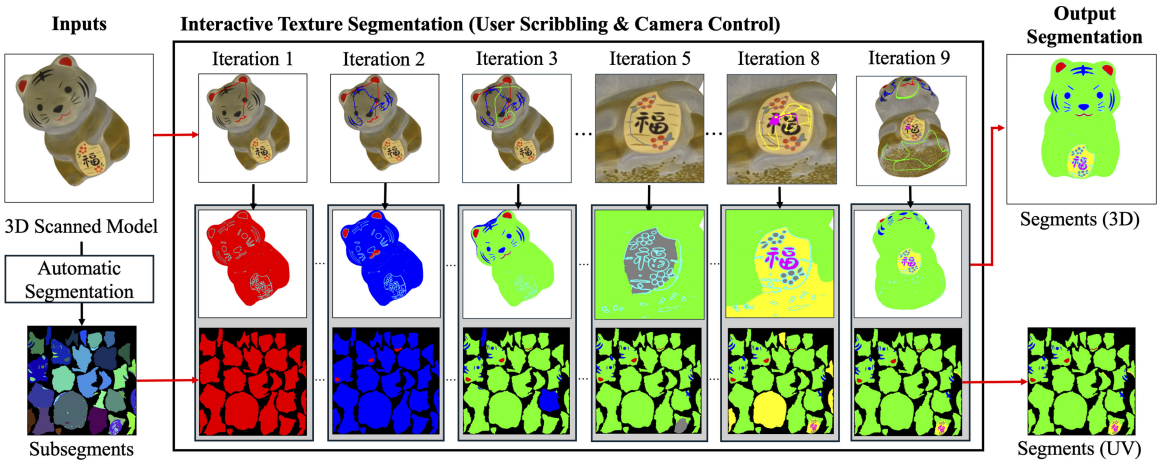


FIGURE 4. Workflow of our interactive texture segmentation using a 3-D scanned model. Automatic segmentation divides the texture into small subsegments in a preprocessing step. The user then iteratively classifies the subsegments into segments by drawing scribbles on the 3-D model. The segment ID of a subsegment is set to the ID of the nearest stroke.

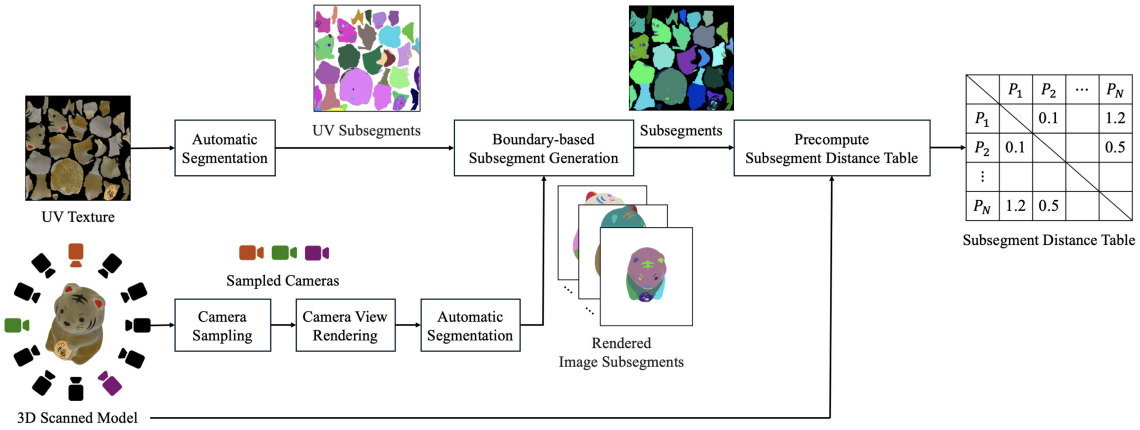


FIGURE 5. Overview of preprocessing. The system first generates two kinds of subsegments, UV subsegments and rendered image subsegments. Then, these two subsegments are merged in boundary-based subsegment generation step. Finally, the system precomputes distances between subsegments to generate a subsegment distance table.

the Supplementary Material. The system first generates two kinds of segmentations: a 3-D view segmentation and a UV segmentation. The system then combines those two segmentations to produce the final segmentation results.

The preprocessing procedure for interactive texture segmentation is as follows.

1) Automatic segmentation of UV texture map.
2) Automatic segmentation of rendered images.
3) Boundary-based subsegment generation.
4) Precomputation of subsegment distance table.

User-guided segmentation is independent from the preprocessing stage. We designed the preprocessing stage to enhance the user experience by accelerating the computation for user scribble inputs during user-guided segmentation.

Furthermore, we want to highlight that boundary-based subsegment generation is assumed to have UV texture and convert it to the small set of segments. On the other hand, similar approach of UV atlassing is packing multiple 3-D model UV maps into a single UV texture to optimize performance and memory usage in rendering.

Authorized licensed use limited to: University of Waterloo. Downloaded on May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.

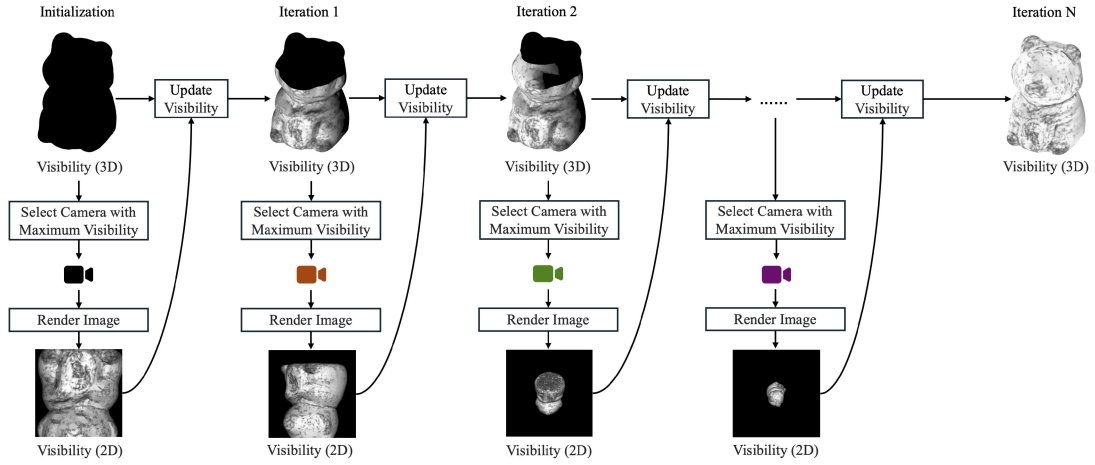


FIGURE 6. Camera sampling. We greedily select a set of cameras that maximizes the visibility of 3-D surfaces.

Preprocessing

Automatic Segmentation of UV Texture Map

We assume a 3-D scanned model and corresponding UV texture as input for the preprocessing stage. The UV texture map consists of disconnected patches. Leveraging the availability of UV texture maps, we apply a 2-D segmentation model to the UV map to obtain a more comprehensive segmentation map. The UV texture is segmented through SAM-HQ[3] as follows:

$$\boldsymbol{S}_{\mathrm{UV}} = \mathrm{SAM}(T_{\mathrm{UV}}) \tag{1}$$

where $\boldsymbol{S}_{\mathrm{UV}}$ denotes a segmentation extracted from the UV texture, SAM denotes a 2-D segmentation model, and T_{UV} denotes a UV texture.

Automatic Segmentation of Rendered Images

Accurate 3-D surface segmentation must account for both shape and appearance. Although 2-D segmentation often achieves higher accuracy and greater flexibility than pure 3-D methods, applying them directly to only a UV texture map ignores important geometric cues. To overcome this limitation, we render the object from multiple viewpoints and run a 2-D segmentation model on each rendered image. By fusing these multiview segmentation outputs, we obtain finer subsegments that faithfully reflect both the object's geometry and its surface texture.

For a given 3-D model, our goal is to select N camera views that collectively cover the entire surface with minimal redundancy. Simply sampling viewpoints on a uniform grid incurs high computational cost and often leaves occluded regions unexplored. Moreover, projecting 2-D masks back onto the mesh can introduce alignment errors along uncertain boundaries in 3-D space. To overcome these issues, we formulate an adaptive camera-selection strategy that iteratively chooses views to maximize incremental surface coverage and visibility quality, as described in the next section.

To address these issues, we adopt an optimization strategy for camera sampling positions, which selects camera views that comprehensively cover the entire 3-D mesh and are oriented forward. When iteratively sampling the camera positions, we use the following two criteria.

1) The candidate view must cover surface regions not yet observed.
2) The candidate view must maximize the visible surface area relative to previously selected views.

The first criterion ensures complete coverage of the 3-D mesh, while the second promotes selection of views with the greatest visibility, facilitating more accurate segmentation. Visibility here refers to the extent to which the mesh surface faces the camera and the size of the visible surface for that view. Figure 6 illustrates an example of camera sampling optimization. In the initialization step, we assume that no area is visible, as no cameras have been sampled yet. In the first iteration, the visibility of the object in 3-D space is updated based on the camera with the best 2-D visibility score. In the second iteration, given the updated 3-D visibility, a new camera is selected to maximize coverage increase. The objective function continuously updates the visibility scores, and cameras are sampled iteratively to improve coverage at each step.

May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.

Once computing the optimal set of cameras $\boldsymbol{C}^*$, we conduct camera view rendering that obtains rendered images $I_{\boldsymbol{C}^*}$ of each camera view. Then, we compute the segmentation results as follows:

$$\boldsymbol{S}_{I_{\boldsymbol{c}_i}} = \mathrm{SAM}\left(I_{\boldsymbol{c}_i}\right) \tag{2}$$

where $\boldsymbol{c}_i$ denotes the ith camera in the optimal set of cameras $\boldsymbol{C}^*$, $S_{I_{\boldsymbol{c}_i}}$ denotes the segmentation result of the ith camera view's image $I_{\boldsymbol{c}_i}$, and SAM denotes the segmentation model (specifically SAM-HQ[3] in our case).

Boundary-Based Subsegment Generation

Merging the segmentation results from different camera views can be difficult since merging is highly correlated with the color information. Instead of merging segments, we combine the *boundaries* from $\boldsymbol{S}_{\mathrm{UV}}$ and $\boldsymbol{S}_{I_{\boldsymbol{C}^*}}$. To transfer 3-D segmentation boundaries into UV-texture space, we first extract the boundary from each camera view and project it onto the UV map. We also encode visibility in the UV map to assess boundary reliability, since segments that are not viewed frontally by the camera are harder to map accurately and can lead to errors in boundary computation. We then binarize the aggregated boundary map, denoted $\mathbf{Bd}_{3d}$, as follows:

$$\mathbf{Bd}_{3d}(x,y) = H\left(\sum_{i=1}^{*} M\Big(\big(|\nabla S_{I_{c_i}}(x,y) V_{c_i}|\big)\Big)\right) \tag{3}$$

where $\nabla S_{I_{c_i}}$ denotes the spatial gradient of the segmentation mask from camera view c_i, M denotes the function that projects each view's boundary map into UV-texture coordinates, H denotes the Heaviside step function, returning 1 if its argument exceeds zero and 0 otherwise, and V_{c_i} denotes the visibility from the camera c_i. We also compute $\mathbf{Bd}_{\mathrm{UV}}$ via $H((|\nabla \boldsymbol{S}_{\mathrm{UV}}|))$. When processing a 3-D segmentation from multiple viewpoints, maintaining boundary consistency is crucial. By incorporating visibility V_{c_i} in (3), we can accurately recover boundaries in highly visible regions where depth cues map cleanly from 3-D space to UV space, whereas low visibility areas remain challenging due to ambiguous camera depth information.

Then, we fuse the UV and 3-D boundary maps via pixelwise addition

$$\mathbf{Bd}^*(u,v) = \mathbf{Bd}_{\mathrm{UV}}(u,v) \; + \; \mathbf{Bd}_{3d}(u,v) \tag{4}$$

where $\mathbf{Bd}^*(u,v)$ denotes the aggregated boundary response at the UV coordinate (u,v), capturing all edges detected in both the UV texture and 3-D projections.

To partition the texture into coherent subsegments, we proceed as follows.

1) *Initial subsegment extraction:* Invert the boundary map

$$\overline{\mathbf{Bd}^*}(u,v) = 1 \; - \; \mathbf{Bd}^*(u,v) \tag{5}$$

   so that interior pixels are 1 and boundaries are 0. Applying connected-component labeling to $\overline{\mathbf{Bd}^*}$ yields a set of preliminary subsegment masks $\mathbf{P}^*_{\mathrm{init}}$.
2) *Boundary label propagation:* For pixels where $\mathbf{Bd}^*(u,v) = 1$, assign a subsegment label by propagating the nearest label from adjacent interior regions via nearest-neighbor interpolation. This produces the final subsegments $\mathbf{P}^*$.

Precomputation of Subsegment Distance Table

Utilizing the subsegments P^* acquired in by boundary-based subsegment generation, we compute the subsegment distance table. First, we create a graph connecting neighboring subsegments with edges. When we create the graph, we define the cost of the edge by the color difference $\times$ Euclidean distance of the two subsegment's center points. Next, we define the distance between all pairs of subsegments using the Dijkstra algorithm. This step is important when determining the cost of the shortest path on the graph at user-guided segmentation. Specifically, the distance calculation assumes that subsegments of similar colors and neighboring locations are likely to be classified within the same segment later during user-guided segmentation.

User-Guided Subsegment Clustering

Our interactive segmentation system clusters subsegments into segments under the guidance of user-provided scribbles. Specifically, the user assigns segment IDs by drawing colored scribbles on the model, and the system propagates these segment IDs to all other subsegments.

For each new scribble operation, the ith subsegment's ID is updated according to the newly added (or removed) scribbles. Subsegments with no scribbles are automatically assigned to the closest scribbled ID using the following equation:

$$k^* = \underset{P_j \cap \boldsymbol{B}_{\mathbf{UV,t}} \neq \emptyset}{\arg\min} \boldsymbol{G}_{P_i,P_j} \tag{6}$$

where k^* denotes the optimal index of the segmentation ID corresponding to the nearest scribbled subsegment, and $P_j \cap \boldsymbol{B}_{\mathbf{UV,t}} \neq \emptyset$ denotes whether

Authorized licensed use limited to: University of Waterloo. Downloaded on May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply. IEEE Limited to University of Waterloo. Downloaded on May 29,2026

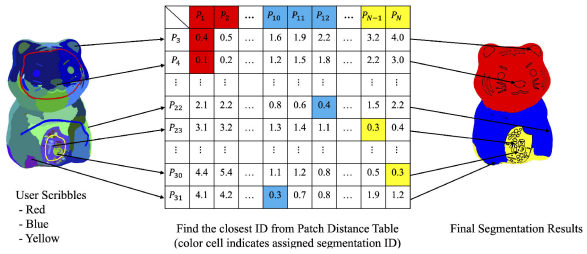


FIGURE 7. Propagation of subsegment IDs. Based on the user scribbles, we assign the segmentation ID to the nonscribbled subsegments by checking the closest user scribble in the subsegment distance table. This yields the final segmentation results.

subsegment P_j is marked in the tth iteration's scribbling map $\boldsymbol{B}_{\mathbf{UV},t}$. Finally, we update the segmentation ID of subsegment P_i to denote it as k^*th segmentation ID col_{k^*}.

Figure 7 illustrates an example of the process. When the segmentation ID is automatically assigned, the ID is first assigned to the subsegments that the user had already assigned. Then, the remaining subsegments are assigned using the subsegment distance table. Specifically, the closest subsegments of already assigned subsegments are determined and the corresponding ID is assigned to the target subsegment.

SUBSEGMENT GENERATION EVALUATION

We then evaluate interactive segmentation quality by segmenting the proposed interface. We are aimed at evaluating the subsegment usability by comparing only UV segmentation by SAM3D.[4]

Experimental Settings

We employ $N = 16$ cameras, with each view segmented by HQ-SAM.[3] Our test models come from Google's publicly available 3-D Scanned Objects Repository.[16] The dataset comprises over 1000 scanned models exhibiting complex textures that must be segmented. From this collection, we selected four particularly challenging examples—featuring character motifs, periodic patterns, and subtle color variations—to demonstrate how our approach enhances subsegment generation both in terms of color consistency and the accurate delineation of character boundaries. To isolate the effect of our merging strategy, we hold both the camera configuration and the HQ-SAM backbone constant, modifying only the algorithm that fuses per-view segmentations. In this way, any performance gap between SAM3D[4] and our method arises solely from the merging approach, not from differences in HQ-SAM's underlying segmentation quality.

For each segmentation task, we manually annotate complete scribbles for every subsegment. We assume that these annotations are close to ideal and place them with precision to ensure clear texture separation via subsegments.

Results of Subsegment Generation

Figure 8 illustrates both the extracted subsegments and the resulting full-image segmentations generated by our interface using the selected four examples. When comparing subsegments, our method faithfully captures fine details—such as the characters in the first row and the pattern texture in the fourth row, whereas SAM3D[4] often produces only coarse, blocky regions. Likewise, in the full-image segmentations, our approach yields cleaner, more accurate separations, while SAM3D's outputs remain too rough for practical interactive editing. In our quantitative evaluation, we define segmentation accuracy as the percentage of visible pixels correctly classified against a manually annotated ground truth. SAM3D achieves an accuracy of $63.48 \pm 10.53\%$, whereas our proposed method achieves $84.60 \pm 5.44\%$, demonstrating a substantial improvement in segmentation quality. Although both systems could benefit from a stronger backbone (e.g., replacing HQ-SAM[3] with an even higher quality model), our merging strategy consistently delivers superior segmentation examples relative to the SAM3D-based pipeline.

USER STUDY

We conducted a user study to assess the segmentation accuracy of our technique and interface compared to a baseline interface utilizing brush painting and a bucket tool. To enable direct comparisons of participants' performance and their experience with each interface, we adopted a within-subjects design. This study received clearance from our institution's ethics review board.

Experimental Settings

Participants

We recruited 16 participants from our institution, all of whom had prior experience with 3-D computer graphics, including gaming and CAD. To ensure this prerequisite was met, we collected information about participants' experiences through a preliminary survey.

Authorized licensed use limited to: University of Waterloo. Downloaded on May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.

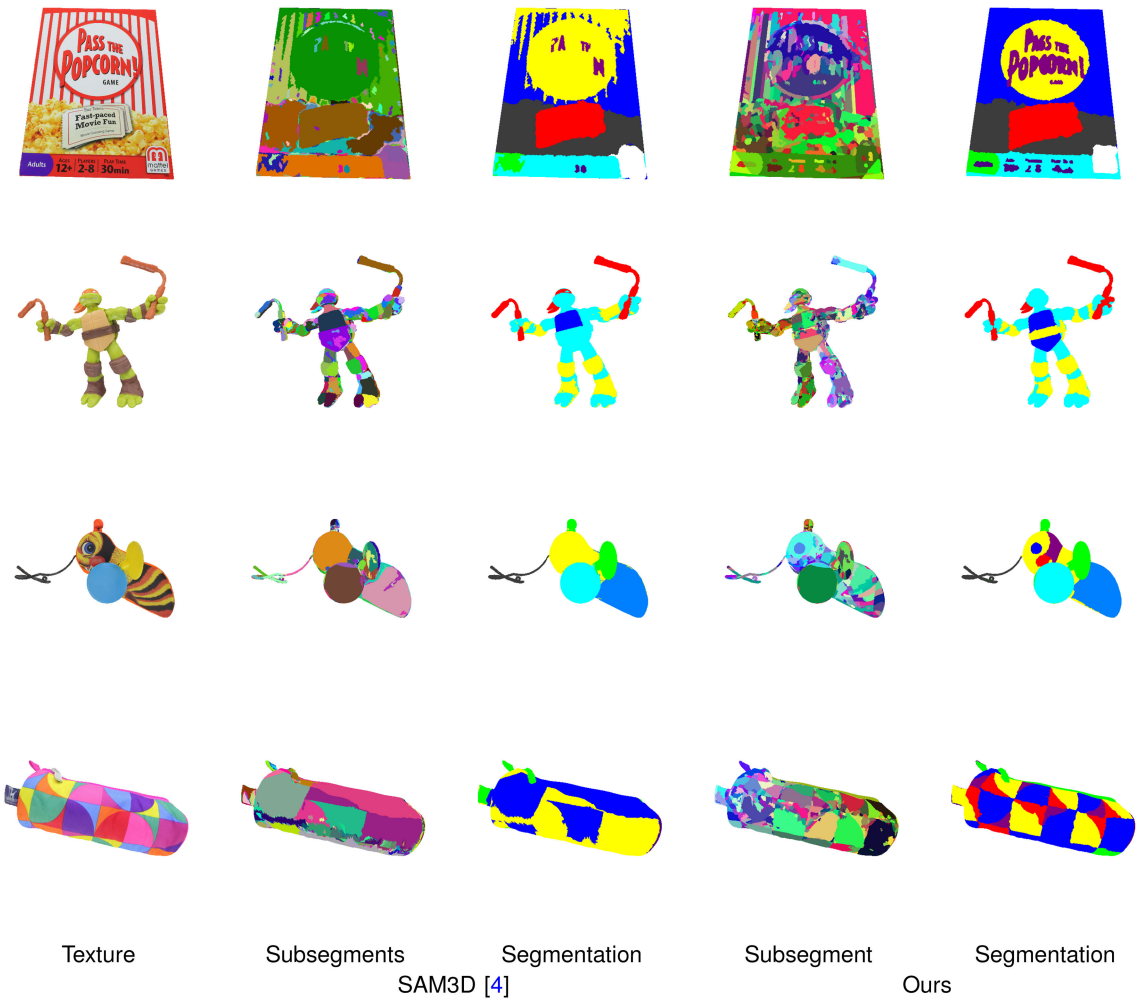


FIGURE 8. Comparison of generated subsegment quality. Our boundary-based method produces higher quality subsegments compared to UV segmentation[3] and SAM3D.[4]

Baseline

Since there is no established automatic process to segment scanned 3-D models, we adopt fully manual brush-based painting as our comparison baseline, which is the baseline technique also used by the material segmentation interface[11] in their evaluation. Specifically, we selected Blender, a widely used 3-D graphics software to create the dataset.[17] Blender's interface allows participants to simultaneously view and manipulate the UV texture and 3-D mesh of models. Texture segmentation in Blender mostly uses brush and fill tools.

Target 3-D Models

The left-hand side of Figure 9 shows the selected models for our study: a broccoli and a watergun, created with a proprietary 3-D scanner. These models were chosen based on two primary characteristics: (1) the presence of indistinct segmentation boundaries, and (2) the existence of occluded or disjointed



FIGURE 9. Target 3-D models of our user study: we assume two models are hard to segment with a limited time constraint.

Authorized licensed use limited to: University of Waterloo. Downloaded on May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.

areas. The broccoli model comprises flower buds and stems. Despite the model's seemingly straightforward structure, the boundaries between the flower buds and stems are not distinctly discernible from various perspectives. In addition, brushing the rear side of the stems presents a significant challenge. The watergun model, despite its comparatively clear color segmentation, introduces its own set of complexities. It features several areas that are difficult to access, including regions around the water tank, trigger, and nozzle. In the preprocessing phase, both models select a relatively large number of camera positions ($N = 32$) from a uniform distribution over the 3-D coordinate space to ensure sufficient coverage, yielding an equal number of SAM outputs for each 3-D view. For the 2-D segmentation model, we used HQ-SAM.[3] To render the 3-D scanned models, we utilized PyTorch3D, setting the rendering resolution of the 3-D view to 1024×1024. To preserve the original image resolution, we set the UV map resolution at 2048×2048. The 2-D segmentation model was adjusted to this resolution to generate matching segmentation results.

Procedure

Each participant took part in the study for approximately 1 h. At the beginning of the session, participants were briefed on the experiment's procedures and asked to provide informed consent. An introductory tutorial on 3-D segmentation was then presented, alongside detailed instructions on how to operate both the proposed and baseline interfaces. Participants were then trained with models (different from those used in the main task) using both interfaces to familiarize themselves with the tools.

For the main study, participants were asked to segment the broccoli and watergun models using each interface (i.e., participants performed four segmentation tasks in total). To mitigate the influence of learning effects on the results, we systematically varied the order in which the interfaces were used by different participants.

Each task had a time limit of 5 min, although participants were allowed to finish earlier if they were satisfied with their segmentation. The objective was to achieve a segmentation that closely resembled a predefined target segmentation. Participants were allowed to refer to the target segmentation for the model at any time during the experiment. While they could ask questions about basic interface operations, inquiries about more efficient strategies for using the segmentation tool were not permitted. In addition, participants were free to select any colors for painting; color differences were not considered in the assessment of their performance.

Evaluation Measure

To evaluate the precision of the segmentation maps created by participants, we compared them with meticulously crafted ground truth maps. These reference maps were created through a rigorous process involving 1–2 h of careful delineation in Blender. We evaluate the accuracy of segmentations as the proportion of correctly classified pixels on the UV map. A pixel was deemed correctly classified if it was marked with the right material type from three possible options or if all material values were zero for regions without an associated material.

User Survey

We assessed the perceived effectiveness of the two user interfaces by asking participants to express their preference on a five-level scale: *strongly prefer baseline*, *prefer baseline*, *neutral*, *prefer proposed*, and *strongly prefer proposed*. In addition, we administered a poststudy questionnaire inspired by the simplified user experience questionnaire (UEQ-S).[18] This survey comprised eight items designed to capture participants' perceptions of the interface's attributes, including its supportiveness, user-friendliness, efficiency, clarity, excitement, engagement, innovation, and state-of-the-art aspects. Participants' responses were recorded on a seven-point Likert scale, where "1" indicated strong disagreement and "7" indicated strong agreement. Furthermore, participants were encouraged to provide open-ended feedback on their experience, highlighting the strengths and weaknesses of the proposed interface, as well as any suggestions for improvement.

Results

Segmentation accuracy: Figure 10 shows a box plot of the accuracy of participant-generated segmentation results. On average, participants achieved an accuracy of $85.95\% \pm 2.75$ for the broccoli 3-D model with our interface, surpassing the baseline's average accuracy of $67.57\% \pm 6.05$. For the watergun 3-D model, our interface resulted in an average accuracy of $95.28\% \pm 0.97$, while the baseline recorded $76.65\% \pm 8.48$. The statistical significance of these differences was confirmed with a Mann–Whitney U test, following a Shapiro–Wilk test suggesting a nonnormal distribution of the results. The significance levels were recorded at $p < 0.01$ for the Broccoli model and $p < 0.01$ for the watergun model (with p-values adjusted using the

May 2026 Authorized licensed use limited to: University of Waterloo. Downloaded on May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.

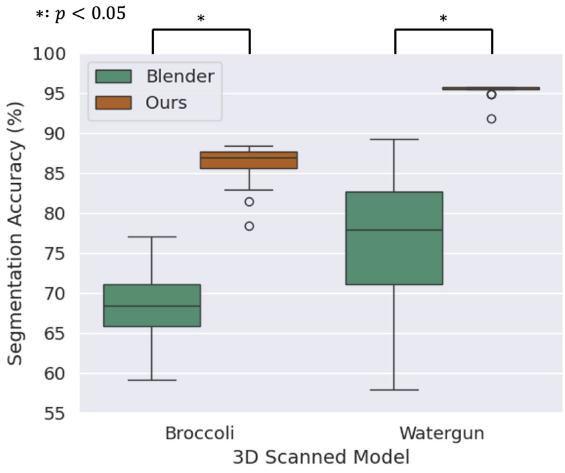


FIGURE 10. Boxplot results illustrate that our approach enhances segmentation accuracy in nearly every instance.

Benjamini–Hochberg procedure for multiple comparisons). These results indicate that participants produced significantly more accurate segmentations for both models using our interface compared to the Blender baseline.

UEQ results: Figure 11 shows the mean scores from the questionnaire. We investigate the user experience of the two interfaces based on the guideline.[18] The pragmatic quality is 1.125 (SD: 1.029) for our approach and −0.188 (SD: 0.864) for Blender. The hedonic quality is 0.938 (SD: 1.206) for our approach and −1.094 (SD: 1.576) for Blender. Overall scores of the simplified UEQ are 1.031 (SD: 0.893) for our approach and −0.641 (SD: 0.907) for Blender. With the benchmark of the guidelines, the respective overall ratings showed that our approach was rated as *above average*, while Blender was rated as *bad*. We confirmed that our approach enhances the overall user experience in performing 3-D segmentation tasks.

Looking more closely at the individual scores, our method achieved better median scores in five categories, but was rated slightly less favorably for "easy" and "clear." Participants familiar with Blender noted that the baseline itself felt more intuitive, and the brush tools more user-friendly. On the other hand, participants who preferred our interface stated that the tool allowed them to speedily segment models with minimal manual intervention. In terms of our interface's clarity, a few participants expressed a preference for manual color application. Nonetheless, these individuals acknowledged the overall convenience of our interface, which contributed to higher scores in various categories when compared to the baseline.

DISCUSSION AND LIMITATION

Importance of 3-D View and Camera Sampling

Figure 12 summarizes the subsegment generation and segmentation results. We obtained the segmentation results by manually clustering on the proposed UI. We pick up one 3-D model from Google's publicly available repository[16] and pick up one characteristic example. We compare three settings: (1) UV-only segmentation, (2) UV segmentation with uniform camera sampling, and (3) UV segmentation with our proposed visibility-aware camera sampling. For each setting, we render two views of the 3-D model to assess segmentation quality in detail. Both sampling-based methods yield finer subsegment granularity compared to UV-only segmentation, while our visibility-aware sampling produces more consistent and coherent texture mappings than uniform sampling. Although uniform sampling can achieve high scores in one view, it also leads to implausible segmentations in another view. In contrast, the proposed method consistently improves subsegment generation and segmentation performance across all evaluated views. Figure 12 also presents the segmentation accuracy, demonstrating that our proposed approach achieves superior quantitative performance.

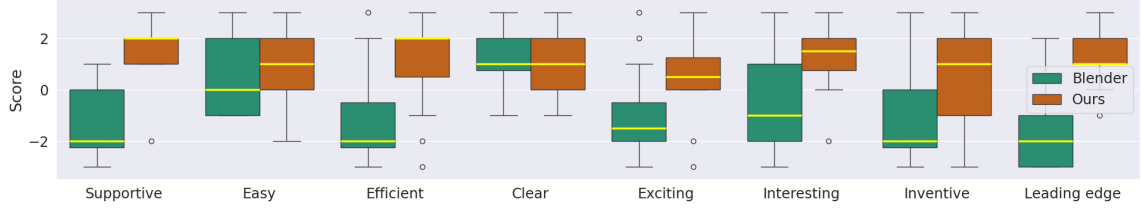


FIGURE 11. Boxplot results derived from the UEQ-S [18] are presented. Users rated their experience on a scale ranging from $[-3, 3]$, with higher scores indicating a more positive assessment of each interface (the yellow line indicates median value). Our method achieved better median scores in all categories but was shown a competitive rating over the "easy" and "clear" categories.

Authorized licensed use limited to: University of Waterloo. Downloaded on May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.

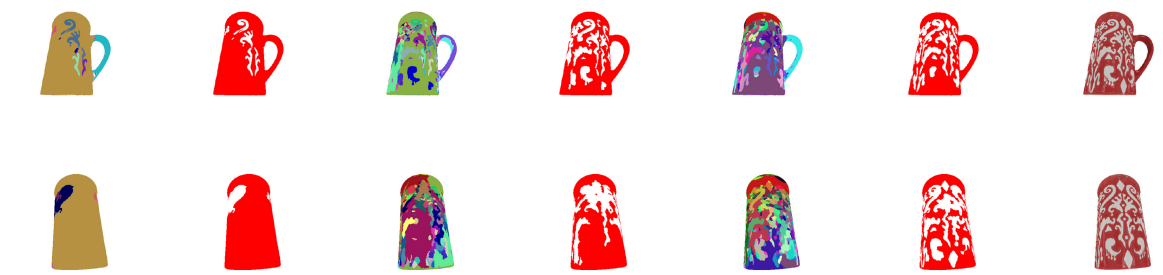


FIGURE 12. Ablation study of camera sampling strategies. We compare subsegment quality and segmentation results under three settings: (1) UV-only segmentation, (2) UV segmentation with uniform camera sampling, and (3) UV segmentation with our proposed visibility-aware camera sampling. Incorporating multiple 3-D camera views significantly improves subsegment detail, and our visibility-aware sampling yields more coherent texture mappings than uniform sampling. We also report segmentation accuracy, defined as the proportion of correctly classified pixels within the visible area.

Manual Subsegment Separation

Our evaluation showed that our technique could effectively divide scanned models into meaningful subsegments. However, our subsegment generation algorithm does not always achieve flawless segmentation, as it may overlook certain segments, such as very small regions or areas sharing similar colors and geometry.

For example, in Figure 12, our final segmentation result is still not perfectly aligned with the UV texture of red and white areas. At the user study, the segmentation accuracy for the watergun model converges to $\sim 95\%$. This suggests that participants were able to achieve high-quality results overall, but minor adjustments with conventional brush painting are still necessary to segment some difficult areas. Hence, support for manual refining of segments at the pixel level is still required to achieve the highest precision. In future versions of our system, we would like to add tools allowing users to manually edit subsegments.

Quality of UV Texture Segmentation

Figure 13 presents the standalone UV-texture segmentation results produced by SAM-HQ.[3] In the first row, SAM-HQ successfully isolates subtle patterns within the texture. However, the second row reveals that smaller regions are not segmented with sufficient precision. We attribute this discrepancy to a domain gap: SAM-HQ is trained on natural image views, which differ markedly from unwrapped UV textures. As a result, its segmentation accuracy on UV maps can be inconsistent. Future work should therefore explore strategies for enhancing segmentation performance, specifically on UV textures.

Boundary Aggregation From 3-D View and UV Texture

Although we accurately align 3-D view boundaries using (3), fully transferring those segmentation

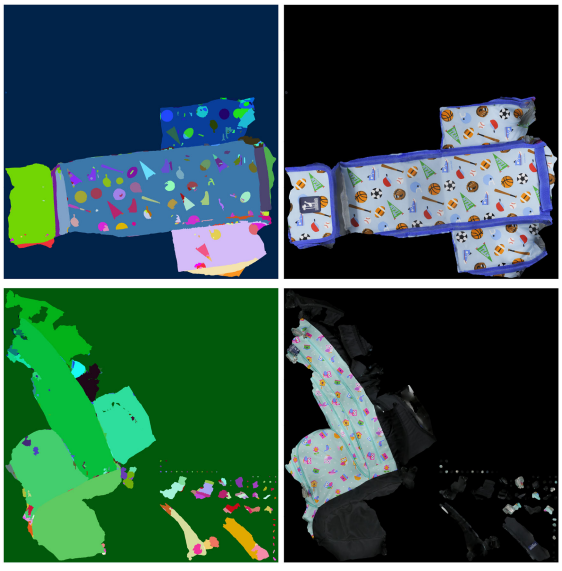


FIGURE 13. UV-texture segmentation results by SAM-HQ[3]: We observe inconsistent segmentation of fine texture details.

May 2026 Author version licensed use limited to: University of Waterloo. Downloaded on May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.

contours into UV space remains challenging due to inherent geometric distortions. In future work, we will develop strategies to resolve these inconsistencies and thereby improve subsegment generation. Moreover, as a practical extension, we envision an interactive interface that allows users to manually refine subsegment boundaries—acknowledging that perfect automatic alignment may not always be feasible.

View-Dependent Nature of Materials

In our system, we only use RGB data of UV textures and camera views since we assume that material information is not available (which is generally the case for models of scanned objects). Therefore, the user can ignore view-dependent material effects, such as reflections and highlights. Estimating material properties from multiple views of the 3-D scanned objects is technically possible, and we believe an interactive system capable of simultaneously performing texture and material segmentation is a promising direction to explore in future work. Another possible enhancement would be to suggest materials based on user-provided segmentation information in addition to the multiple views of the 3-D scanned model. High-quality automatic material segmentation remains a challenge.[19] Therefore, at this stage, our system and study focus solely on texture segmentation.

CONCLUSION

In this article, we proposed an interactive texture segmentation method for 3-D scanned models, combining automatic segmentation with minimal manual input to balance efficiency and user control. In a preprocessing phase, the technique identifies small subsegments in the model, which are subsequently clustered by users using intuitive scribbles on the 3-D model view. Evaluations confirmed that our method significantly improves segmentation accuracy and quality compared to conventional manual techniques using standard 3-D computer graphics software. In addition, our method produces more detailed segments compared to fully automatic algorithms that segment UV texture maps. The main limitation of our technique is the extended preprocessing time required to precompute the subsegments.

ACKNOWLEDGMENTS

Our user evaluation was conducted in accordance with our institution's ethical review regulations.

REFERENCES

1. J. MacQueen et al., "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Probability*, Oakland, CA, USA, 1967, pp. 281–297.
2. R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.
3. L. Ke et al., "Segment anything in high quality," in *Proc. 37th Int. Conf. Neural Inf. Process. Syst.*, 2023, pp. 29914–29934.
4. Y. Yang, X. Wu, T. He, H. Zhao, and X. Liu, "SAM3D: Segment anything in 3D scenes," 2023, *arXiv:2306.03908*.
5. H. Guo et al., "SAM-guided graph cut for 3D instance segmentation," in *Proc. Eur. Conf. Comput. Vis.*, 2024.
6. K. Madono et al., "Interactive texture segmentation of 3D scanned models leveraging multiview automatic segmentation," in *Proc. Extended Abstr. CHI Conf. Hum. Factors Comput. Syst.*, New York, NY, USA, 2025, pp. 1–7. [Online]. Available: https://doi.org/10.1145/3706599.3719697
7. M. Landy and J. A. Movshon, "The plenoptic function and the elements of early vision," in *Computational Models of Visual Processing*. Cambridge, MA, USA: MIT Press, 1991, pp. 3–20.
8. B. Mildenhall, P.P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, pp. 99–106, 2021.
9. B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graphics*, vol. 42, no. 4, pp. 1–14, 2023.
10. T. Igarashi, N. Shono, T. Kin, and T. Saito, "Interactive volume segmentation with threshold field painting," in *Proc. 29th Annu. Symp. User Interface Softw. Technol.*, New York, NY, USA, 2016, pp. 403–413. [Online]. Available: https://doi.org/10.1145/2984511.2984537
11. W. Kawabe, T. Hashimoto, F. Matulic, T. Igarashi, and K. Higuchi, "Interactive material annotation on 3D scanned models leveraging color-material correlation," in *Proc. SIGGRAPH Asia 2023 Tech. Commun.*, New York, NY, USA, 2023, Art. no. 18. [Online]. Available: https://doi.org/10.1145/3610543.3626170
12. H. E. Wong, M. Rakic, J. Guttag, and A. V. Dalca, "ScribblePrompt: Fast and flexible interactive segmentation for any biomedical image," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 207–229.

Authorized licensed use limited to: University of Waterloo. Downloaded on May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.

13. F. Chevalier, P. Dragicevic, and C. Hurter, "Histomages: Fully synchronized views for image editing," in *Proc. 25th Annu. ACM Symp. User Interface Softw. Technol.*, New York, NY, USA , 2012, pp. 281–286. [Online]. Available: https://doi.org/10.1145/2380116.2380152
14. Y. Boykov and M.-P. Jolly, "Interactive graph cuts for optimal boundary & region segmentation of objects in N-D images," in *Proc. 8th IEEE Int. Conf. Comput. Vis.*, 2001, pp. 105–112.
15. D. Sýkora, J. Dingliana, and S. Collins, "LazyBrush: Flexible painting tool for hand-drawn cartoons," *Comput. Graphics Forum*, vol. 28, no. 2, pp. 599–608, 2009.
16. L. Downs et al., "Google scanned objects: A high-quality dataset of 3D scanned household items," in *Proc. 2022 Int. Conf. Robot. Autom.*, 2022, pp. 2553–2560.
17. A. I. Károly and P. Galambos, "Automated dataset generation with blender for deep learning-based object segmentation," in *Proc. IEEE 20th Jubilee World Symp. Appl. Mach. Intell. Informat.*, 2022, pp. 000329–000334.
18. M. Schrepp, A. Hinderks, and J. Thomaschewski, "Design and evaluation of a short version of the user experience questionnaire (UEQ-S)," *Int. J. Interact. Multimedia Artif. Intell.*, vol. 4, pp. 103–108, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID
19. P. Sharma, J. Philip, M. Gharbi, B. Freeman, F. Durand, and V. Deschaintre, "Materialistic: Selecting similar materials in images," *ACM Trans. Graphics*, vol. 42, no. 4, Jul. 2023, Art. no. 154. [Online]. Available: https://doi.org/10.1145/3592390

KOKI MADONO is an assistant professor at the Department of Creative Science, Waseda University, Tokyo, 169-8555, Japan. His research interests include interactive computer graphics and rendering system especially for ink painting. He is the corresponding author of this article. Contact him at madonomadonorunning@gmail.com.

TAKEO IGARASHI is with Preferred Networks Inc., Tokyo, 100-0004, Japan. He is also a professor at the Department of Creative Informatics, The University of Tokyo, Tokyo. His research interests include user interfaces and interactive computer graphics. Contact him at takeo@acm.org.

HIROHARU KATO is a researcher at Preferred Networks Inc., Tokyo, 100-0004, Japan, and leads the development of the 3-D scanning service "PFN 3D Scan." His research interests include 3-D vision, computer vision, and computer graphics. Contact him at hkato@preferred.jp.

TAISUKE HASHIMOTO is an engineer at Preferred Networks Inc., Tokyo, 100-0004, Japan, and leads the development of the 3-D scanning service "PFN 3D Scan." His research interests include 3-D vision and computer graphics. Contact him at hashimotot@preferred.jp.

FABRICE MATULIC is a senior researcher at Preferred Networks Inc., Tokyo, 100-0004, Japan, and an assistant professor at the University of Waterloo, Waterloo, Canada. His research interests include interactive surfaces and pen computing, embodied and remote interaction with large displays, and extended reality. Contact him at fmatulic@preferred.jp.

TSUKASA TAKAGI is an engineer at Preferred Networks Inc., Tokyo, 100-0004, Japan, and leads the development of the 3-D scanning service "PFN 3D Scan." His research interests include 3-D vision and computer graphics. Contact him at takagi@preferred.jp.

KEITA HIGUCHI is a researcher at Preferred Networks Inc., Tokyo, 100-0004, Japan, and cofounder at Jokun Brewing Lab, Niigata, Japan. His research interests include multimedia interfaces, accessibility, and HCI for machine learning. Contact him at khiguchi@acm.org.

May 29,2026 at 13:09:01 UTC from IEEE Xplore. Restrictions apply.